



Before & After: Data Flow Comparison

THE PROBLEM: Data Reset on Restart

What Was Happening (BEFORE - BROKEN)

RAILWAY RESTART SEQUENCE

Timeline:

T=0min

⏏ Railway Restarts ⏏ (e.g., deploy, crash, timeout)



T=1min

⏏ Container starts with fresh git repo ⏏
⏏ (includes original users.json from ⏏
⏏ repository history) ⏏



T=2min

⏏ bot.js executes: ⏏
⏏ **const** users = loadUsers(); ⏏
⏏ // ❌ READS OLD users.json ⏏



```
users = {
  8349475987: {
    credits: 50, ❌ THIS IS OLD DATA!
    history: [...]
  },
  ...
}
```



T=3min

⏏ cancelPendingOrders() executes: ⏏
⏏ **for** (**const** [id, user] of users) { ⏏
⏏ syncUserToSupabase(id, user) ⏏
⏏ // ❌ OVERWRITES SUPABASE! ⏏
⏏ } ⏏



T=4min

⏏ SUPABASE UPDATED: ⏏
⏏ User 8349475987 credits: 150 ➡ 50 ❌ ⏏
⏏ (All new data overwritten with old data) ⏏
⏏ User loses 100 credits! 💣 ⏏

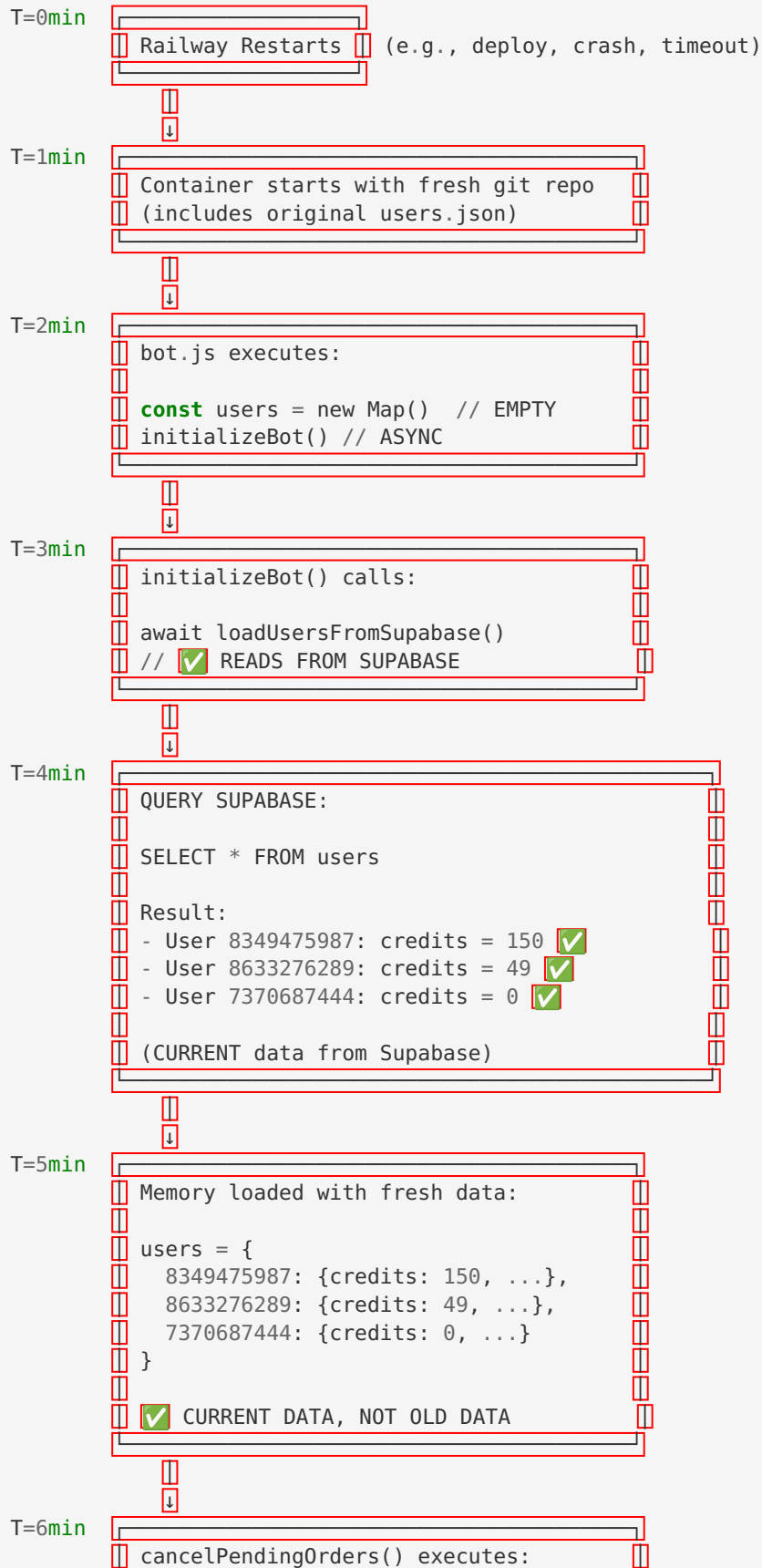
RESULT: Users lose credits, logs disappear ❌

THE SOLUTION: Load from Supabase First

What Happens Now (AFTER - FIXED)

RAILWAY RESTART SEQUENCE

Timeline:



```

// ✅ ONLY CLEANS 5SIM ORDERS
// ✅ DOES NOT SYNC DATA
Result: 0 orders cancelled

```

T=7min

```

BOT READY:
✅ All users loaded with current credits
✅ No data lost
✅ Supabase data intact
Users keep their credits! 🎉

```

RESULT: Users keep credits, Supabase data intact ✅

Detailed Comparison: Code Flow

BEFORE (Broken Flow)

```

// ❌ Step 1: Load from JSON (stale data from git repo)
const users = loadUsers();
// users now contains OLD data from users.json

// ❌ Step 2: Sync old data to Supabase (overwrites!)
async function cancelPendingOrders() {
  for (const [id, user] of users.entries()) {
    syncUserToSupabase(id, user); // OVERWRITES SUPABASE!
  }
}
cancelPendingOrders();

// Result: Supabase overwritten with stale JSON data 💣

```

AFTER (Fixed Flow)

```
// ✅ Step 1: Start with empty map (no stale data)
let users = new Map();

// ✅ Step 2: Load from Supabase first (current data)
async function initializeBot() {
  // Primary: Load from Supabase
  users = await loadUsersFromSupabase();

  // Fallback: If Supabase fails, use JSON
  // (loadUsersFromSupabase handles this internally)

  // Only clean up 5sim orders, don't sync data
  await cancelPendingOrders();
}

initializeBot();

// Result: Supabase data preserved ✅
```

Data Source Priority

BEFORE (Wrong Priority)

User Data Sources (Priority):

- ❌ users.json (PRIMARY - WRONG!)
- ❌ Supabase (overwritten by JSON)

Problem: JSON always overwrites Supabase

AFTER (Correct Priority)

User Data Sources (Priority):

- ✅ Supabase (PRIMARY - CORRECT!)
- ✅ users.json (FALLBACK ONLY)

Benefit: Supabase is source of truth

Function Changes Summary

Function	Before	After	Impact
<code>loadUsers()</code>	Loads JSON as primary	Renamed to <code>loadUsersFromJSON()</code> , fall-back only	Prevents stale data
<code>loadUsersFromSupabase()</code>	Didn't exist	NEW: Loads from Supabase	Primary data source
<code>initializeBot()</code>	Didn't exist	NEW: Async initialization	Ensures Supabase loads first
<code>cancelPendingOrders()</code>	Synced data to Supabase	Only cleans 5sim orders	Prevents data overwrites
Bot startup	Synchronous	Asynchronous	Allows Supabase query to complete

Impact on User Workflow

User Scenario: Adding Credits

BEFORE (With Bug)

Time	Action	Result
1:00	Admin: <code>/addcredits 8349475987 50</code> Bot: Updates memory, syncs to Supabase Supabase: User now has 150 credits	
2:30	Railway auto-restarts (deploy) Bot: Loads <code>users.json</code> (has 100 credits) Bot: Syncs old data to Supabase ✨ Supabase: User now has 100 credits ❌	
	User lost 50 credits! 😞	

AFTER (Fixed)

Time	Action	Result
1:00	Admin: /addcredits 8349475987 50 Bot: Updates memory, syncs to Supabase Supabase: User now has 150 credits	
2:30	Railway auto-restarts (deploy) Bot: Loads from Supabase (has 150 credits)  Bot: Cleans 5sim orders only Supabase: User still has 150 credits 	
User keeps their 50 credits! 🎉		

Testing the Fix

Test Scenario: Verify Data Persistence

BEFORE (Broken):

1. Add 50 credits → User has 100
2. Restart bot
3. Check credits → User has 50 
(Reset to old value)

AFTER (Fixed):

1. Add 50 credits → User has 100
2. Restart bot
3. Check credits → User has 100 
(Preserved from Supabase)

Database Consistency

BEFORE: Inconsistent

users.json (JSON file):
User 8349475987: 100 credits

Supabase (Database):
User 8349475987: 150 credits

Status:  INCONSISTENT (JSON is stale)

On restart: JSON overwrites Supabase → Data loss

AFTER: Consistent

Supabase (Database - Source of Truth):
User 8349475987: 150 credits

users.json (Local backup):
User 8349475987: 100 credits (outdated but safe)

Status:  CONSISTENT (Supabase is authoritative)

On restart: Loads from Supabase → No data loss

Key Improvements

Aspect	Before	After
Data Source	Stale JSON	Current Supabase
On Restart	Load JSON, lose data	Load Supabase, keep data
Sync Logic	Overwrite Supabase	Never overwrite
Fallback	None (JSON was primary)	JSON (if Supabase fails)
User Impact	Credits lost on restart	Credits preserved
Data Safety	 Low	 High
Reliability	 Unstable	 Reliable

Expected Behavior After Fix

-  User adds credits
-  Credits appear in admin panel
-  Bot restarts (Railway deploy, crash, etc.)
-  User credits still there (not reset)
-  Admin panel shows same credits
-  Supabase has current data

This is the correct behavior that this fix implements!

Visual comparison created to help understand the bug and solution